

By GWANHOO LEE, WILLIAM DELONE, and J. ALBERTO ESPINOSA

AMBIDEXTROUS COPING STRATEGIES IN GLOBALLY DISTRIBUTED SOFTWARE DEVELOPMENT PROJECTS

Strategies for enhancing flexibility and rigor.

Software development increasingly requires globally distributed teams as organizations seek to deliver high-quality software to global users and customers at lower development costs [10]. This globalization increases the complexity of coordination in the collaborative software development effort, which can in turn negatively influence project outcomes [6]. Geographic distance, time separation, cultural differences, language differences, organizational boundaries, and functional boundaries inherent in global contexts represent significant barriers for global software teams [4, 7, 9].

Thus, a key challenge for information systems organizations today is to overcome these global boundaries and barriers and deliver high-quality software on time and within budget. However, coping with this challenge can be a daunting task because organizations have a limited understanding of what makes global software development projects successful.

To shed light on how organizations cope with global boundaries and barriers to succeed in software development, we studied 22 globally distributed software projects. Similar to prior studies of global teams [10], we analyzed key inputs, processes, and outputs of interest to our study (see the sidebar on the next page for our research methodology). The main inputs of interest are the various global boundaries and barriers that software teams faced. The processes we studied are the task processes involving communication and coordination that are generally employed by software teams [6]. Finally, the outputs we focused on include software quality and on-time/within-budget completion that are important success measures in software projects.

Global boundaries and barriers hinder the efficiency and effectiveness of traditional task processes employed in software projects, which can negatively affect project outcomes, making it more difficult to succeed in globally distributed projects than in co-located projects. In this research, we found that effective global software teams adopted special coping strategies to handle the difficulties of global contexts and to mitigate their negative effects on project outcomes. Consistent with prior studies, global teams tailored task processes and project setups to fit the global context [5]. More importantly, we found that effective coping strategies used by global software teams exhibited “ambidextrous” properties [1, 2]; that is, the coping strategies established rigor and discipline in software development activities while they also built flexibility and agility to quickly sense important external environmental changes and respond to them in a timely manner.

EFFECTIVE COPING STRATEGIES FOR GLOBALLY DISTRIBUTED SOFTWARE DEVELOPMENT

We organized the coping strategies we identified in

Timing	Focus	Coping Strategies
Initiation phase	Task	Common Platform - Shared domain/application knowledge - helped offset potential communication problems. - helped plan project activities with reduced communication. - Common task process - common software development processes and project management. - common working/management styles, clear delineation of responsibilities. - Common task context and work environment - helped create realistic assumptions about how work is done in other sites.
	People	Labor Organization - Minimized task dependencies - an independent portion of tasks was assigned to people in the same location to minimize dependencies with other locations. - Redundant roles/point persons - arranged redundant roles and presence across multiple locations. - prevented a single point of failure; increased responsiveness to issues “Follow-the-sun” model 24-hour project operation by turning tasks over to other sites in different time zones. Education/Understanding - Global lessons learned - shared the knowledge base of lessons learned from past global projects. - Planning workshop - face-to-face workshops to promote common, deep understanding of global boundaries and barriers and their impacts on project performance.
	Technology	Technology Readiness - Technology infrastructure - installed compatible technological environments across global sites. - Integrated project management tool - integrated project management tools were installed. - Portfolio of collaboration tools - multiple collaboration tools were provided.
Execution phase	Task	Doing More - More and continuous communication - the global communication channel was open on the 24/7 basis. - Extended work hours/shifts - non-conventional work shifts, longer hours, coordinated vacation schedules. - helped deal with time zones and ensured synchronous communication. - More and detailed documentation - use of more diagrams, careful validation of user requirements. - detailed project logs for conference calls and meetings. - Tight project control - more progress reports, project reviews, debriefings, conference calls. - frequent regular meetings for corrective action.
	People	Awareness/Teamwork - Team awareness - developed awareness of the expertise of others, co-located part of the team, rotated personnel across sites, developed trust among teammates. - helped teams cope with time and distance separation. - Task awareness - maintained task awareness through frequent visits to other sites and use of shared project documents and project management tools.
	Technology	Adaptive Use Of Technology - Broad usage - multiple tools and technologies were used to maximize communication frequency and effectiveness. - instant messaging, teleconferencing, videoconferencing, Web meetings, extranets, intranets. - Evolutionary usage - kept technology that worked and dropped technology that did not work. - adopted the technologies that worked best for specific tasks.

Table 1. Coping strategies for globally distributed software development.

our interviews using a two-dimensional framework that emerged from our data analysis (see Table 1). The first dimension of the framework captures the timing of these strategies in terms of whether they were applied in the initiation phase or in the execution phase of the project life cycle. The second dimension indicates if the focus of a coping strategy is on task-related processes, people-related project aspects, or adoption and use of collaboration technology. For each of the six cells in the framework, we inductively grouped specific coping strategies into categories that emerged from our data. We named

these categories *Common Platform*, *Labor Organization*, *Education/Understanding*, *Technology Readiness*, *Doing More*, *Awareness/Teamwork*, and *Adaptive Use of Technology*.

Common Platform refers to any frame of reference that provided a common ground to foster a shared understanding of how the software development task will be performed. Labor Organization means the special arrangements through which global software teams assigned tasks and team members across global locations. Education/Understanding includes the activities that increased team members' understanding of unique challenges in global software development as well as effective coping strategies learned from past experience. Technology Readiness refers to the technological setup that facilitated global collaboration. Doing More refers to the increased frequency and intensity of software development activities and processes. Awareness/Teamwork means the processes and strategies that made a globally distributed team work like a co-located team by increasing awareness of who is doing what and by building trust. Finally, Adaptive Use of Technology refers to the evolutionary use of collaboration technology as software teams learned about which technology was best suited to which task. The specific coping strategies shown in Table 1 represent the most frequent examples in each category that we found in our field study.

The two-dimensional framework and specific coping strategies shown in Table 1 provide a comprehensive and integrative perspective that can help global software project managers to plan and implement effective coping strategies at different stages of the project life cycle. Although they are not meant to be exhaustive, these coping strategies can serve as an initial portfolio that practitioners may adopt in order to

identify and select coping strategies that are effective for their specific global project contexts.

Although the coping strategies were found to enhance project success, their utilization came with substantially more effort, stress, and burden on the project teams. One project manager we interviewed said, "I would think that it would be foolish to not take whatever mathematical calculation you have (for a domestic project) and start off with 1.5 times that budget and plan amount." Some project teams established a 24/7 global communication channel that demanded exceptional time commitments and resulted in high levels of personal stress. Another project manager stated, "It [our communication channel] was always open for any issue. This was a must and not an option. The real communication between the programmers there and the users here was something that had to be constantly going on." Increased project costs also resulted from creating redundant project roles and point persons in multiple locations.

Some global software teams were able to plan and implement effective coping strategies from the inception of their projects whereas other teams deployed coping strategies at later points in their projects but only after experiencing painful and costly problems. Several teams in our study experienced temporary failure at some point in time, but they managed to turn around their projects by adopting effective coping strategies. We found that prior global project experience makes a difference in how early in the project life cycle the project team was able to identify and apply effective coping strategies. Teams with extensive prior global project experience tended to implement effective coping strategies at the start of their projects. However, teams with little or no global project experience did not deploy effective measures until they

HOW THE STUDY WAS CONDUCTED

We collected the data for this study through one-hour, semi-structured, face-to-face/telephone interviews with 22 managers of large global software development projects from seven different companies operating in Australia, India, Ireland, Mexico, South Africa, the U.K., and U.S. These seven organizations represented automotive, music, computer, financial, and IT services industries.

All participants had significant project management responsibilities for their projects, ranging from project portfolio management to development lead. Six participants were IT executives and other 16 participants were managers. On average, they had 6.6 years of work experience managing global software projects. The software development budgets for these projects ranged from approximately \$500,000 to \$45 million with the mean of \$11 million.

We asked them to provide detailed descriptions on how various global boundaries and barriers affected their project outcomes and which coping strategies were most effective in mitigating the negative effects of these global factors. The interviews were audiotaped and transcribed verbatim. The text was then analyzed using open and axial coding, following a grounded theory approach [12]. During open coding we identified recurrent themes discussed in the interviews and during axial coding we found how the various themes related to each other. More specifically, we focused on finding causal attributions on how global boundaries and barriers affected project outcomes and how various coping strategies helped mitigate the respective negative effects of these factors. **C**

faced failure during the execution of the project.

As organizations engage in more global projects, we would expect them to better manage global challenges if they develop and maintain a project repository where they accumulate and share global project experience and knowledge. A manager in our study emphasized, “So the advice that I would give is, in ensuring the success of the project going forward, to make sure the lessons we learned, which is all the domain knowledge that we have gained, not to be lost.”

The two coping strategies within the Labor category, namely minimized task dependencies and the follow-the-sun model, might appear to be contradictory. The follow-the-sun model might seem to require tight dependency between multiple locations. However, analysis and interpretation of the interviews suggest they are not actually contradictory. We found the follow-the-sun model worked well either when there was little task dependency, or when the management of task dependencies could be easily programmed and automated in the direction of the workflow. Therefore, minimizing task dependencies can facilitate the implementation of the follow-the-sun model. Furthermore, we found that simple, unambiguous, well-defined tasks fit the follow-the-sun model whereas complex, ambiguous, ill-defined tasks did not fit the model as well.

One of our most important findings is that successful global software development required not only flexibility/agility but also rigor/discipline in order to cope with complex challenges of global projects. For example, once certain strategies were adopted, the team had to comply with these strategies in a disciplined and rigorous way. At the same time, the team had to exhibit flexibility to adapt quickly to revised strategies when needed.

AMBIDEXTERITY OF COPING STRATEGIES

Effective software teams demonstrated an ambidextrous combination of flexibility and rigor rather than sole focus on flexibility or rigor. Table 2 highlights how each category of the coping strategies from Table 1 contributed to building flexibility or rigor in global software development. These results

Coping Strategies	Contribution to Software Development	
	Flexibility	Rigor
Common Platform	- Enabled software teams to make changes less costly and well orchestrated across global locations.	- Helped ground communications and improved effectiveness of information exchange. - Reduced ambiguity in documents, scope, and responsibilities.
Labor Organization	- Minimized task dependencies resulting in loosely coupled global sites. - A point person in offshore sites helped quickly identify and resolve issues.	- Redundant roles prevented single points of failure and point persons closed communication gaps. - The follow-the-sun model increased productivity.
Education/ Understanding	- Helped teams sense and interpret external environmental changes. - Made teams less resistant to adaptation.	- Teams' knowledge about global issues and effective coping strategies reduced mistakes and errors.
Technology Readiness	- Having a portfolio of collaboration tools allowed teams to choose effective tools later as the project progressed.	- Software was developed and tested in multiple locations under the same technological environment. - The project was managed by integrated project management tools.
Doing More	- Frequent, extensive communication helped teams to sense and respond to problems and issues in real time.	- Detailed, accurate documentation minimized ambiguity/misunderstanding - Tighter project controls ensured projects stayed on track.
Awareness/ Teamwork	- A “one team” mind-set and trust across sites facilitated smooth coordination for making changes.	- Reduced errors and mistakes through increased team/task awareness. - Increased efficiency in coordination.
Adaptive Use of Technology	- Effective technologies for specific contexts were adopted over time. - Optional technologies were available.	- Specific technologies were dedicated to specific tasks and purposes to maximize efficiency and effectiveness.

Table 2. Coping strategies for flexibility and rigor in software development.

help managers develop a deeper understanding of the mechanisms through which coping strategies enhance ambidexterity in software development.

All seven categories shown in Table 2 improved both flexibility and rigor. Furthermore, we found that specific coping strategies within the same category often played different roles in increasing flexibility or rigor; some strategies primarily contributed to flexibility and others mainly contributed to rigor. For example, within the Labor Organization category, the minimized task dependencies strategy resulted in loosely coupled global sites, thus increasing flexibility, whereas redundant project roles in multiple sites prevented single points of failure, increasing rigor and reliability of global software development.

CONCLUSION

We found that successful global software teams applied common principles in deploying coping strategies to enhance both flexibility and rigor in software development. Specifically, the following three general principles emerged from our analysis and interpretation of the research interviews.

The first principle is that, in the initiation phase of projects, *teams need to build a foundation for future flexibility in global software development*. Common, standardized processes and shared knowledge/context established at the initiation of the project enable teams to make changes later in the project at a lower cost and in a more coordinated manner.

COPING STRATEGIES CAN SERVE AS

AN INITIAL PORTFOLIO THAT PRACTITIONERS MAY ADOPT IN ORDER TO IDENTIFY AND SELECT COPING STRATEGIES THAT ARE EFFECTIVE FOR THEIR SPECIFIC GLOBAL PROJECT CONTEXTS.

Teams also increase future flexibility by embedding options into the initial project setup. For example, they establish a portfolio of various communication and collaboration technology options from which they can choose based on what works or doesn't work for a particular stage of the project. Education plays an important role as well. As software team members develop a common, deep understanding of potential issues and problems associated with global boundaries and barriers, they become more effective in sensing and interpreting external environmental changes and less resistant to adaptation in the execution phase. Finally, the special labor organization that minimizes task dependencies across sites makes local changes less costly because no cascading changes are required by other sites.

The second principle is that *effective global software teams flexibly deploy coping strategies as needed during the execution phase of projects*. Since many external changes are unpredictable, rapid sensing and response becomes a key coping mechanism. Effective software teams are able to sense important environmental changes quickly and make timely responses. Frequent communication and increased team/task awareness help global software teams to effectively sense and respond to problems and issues. For example, the "24/7 command centers," "project dashboards," "weekly corrective action meetings," and use of point people play pivotal roles in sensing and responding to emergent problems on a real-time basis. Global software teams also learn and adopt the technologies that best fit specific tasks and, at the same time, quickly eliminate the use of technologies that do not work.

The third principle is that, while building and maintaining flexibility is critical, *successful global software teams also exhibit disciplined adherence to the agreed-upon strategies and processes*. Teams with global boundaries and barriers require rigorous software development processes because people cannot always communicate frequently and spontaneously to make adjustments when members depart from the agreed-upon processes.

For example, detailed, accurate documentation and user requirement specifications reduce ambiguity and misunderstanding. Redundant roles in multiple locations reduce the presence of single points of failure. Assigning point persons to offshore sites improves the effectiveness of communication. Common technological environments also allow global software teams to develop and test software efficiently. These rigorous, disciplined software development processes serve as the foundation for positive project performance.

Because the coping strategies we identified in this research are intended to enhance not only flexibility but also rigor in software development, some of them may seem inconsistent with the conventional agile software development approach that focuses primarily on flexibility. According to the *Agile Manifesto and Principles*, the conventional agile software development values people over processes/tools, working software over comprehensive documentation, customer collaboration over contract negotiation, and responding to change over following a plan [3]. Furthermore, in agile software development tacit knowledge is more important than explicit knowledge and informal communication is more useful than traditional formal communication [8].

These agile principles appear to be incompatible with such coping strategies identified in this study as detailed, formal documentation, emphasis in common, disciplined processes, and tight project control. The conventional agile software development approach, however, has been developed and operated best in small and collocated projects. We typically see difficulties arise in attempting to scale agile methods to fit large or globally distributed software development projects [11]. In particular, global boundaries and barriers such as time separation and geographic distance significantly increase the complexity of software development activities, making the conventional agile approach less effective. Our research with globally distributed projects helps shed some light on this challenge.

OUR FINDINGS ARE NOT CONTRADICTIONARY

TO THE AGILE APPROACH, BUT RATHER THEY EXTEND

AGILE METHODS BY DEMONSTRATING HOW FLEXIBILITY SHOULD

BE PARTNERED WITH RIGOR AND DISCIPLINE IN ORDER TO ACHIEVE

SUCCESS IN GLOBAL SOFTWARE DEVELOPMENT PROJECTS.

Our findings suggest that for globally distributed projects, the conventional agile methods must be adjusted and modified by embracing more rigor and discipline in software development. In small, collocated projects, rigorous, disciplined software development processes may hinder agility and flexibility. However, without rigor and discipline, software development can become chaotic and inefficient in globally distributed environments. For example, common, rigorous processes become important when multiple global boundaries exist because people cannot easily substitute for processes as they can in small, local projects.

Detailed, comprehensive documentation as well as codified, explicit knowledge are critical in global contexts because communication is problematic and tacit knowledge is difficult to share. In addition, in globally distributed software development environments, formal communication is also important because informal communication is often less effective due to cultural differences, language barriers, and organizational boundaries.

Based on our findings we propose that the conventional agile software development approach must be modified in globally distributed environments to overcome the daunting challenges of time separation, geographic distance, and cultural differences. The coping strategies identified in tables 1 and 2 can help adapt the conventional agile methods for global projects by pursuing both flexibility and rigor. Therefore, our findings are not contradictory to the agile approach, but rather they extend agile methods by demonstrating how flexibility should be partnered with rigor and discipline in order to achieve success in global software development projects.

In conclusion, while information systems managers who lead globally distributed software development projects must be forewarned that global boundaries and barriers increase project risks, the application of coping strategies that enhance both flexibility and rigor in complex global projects can

help global teams meet their goals and succeed. Understanding how to achieve ambidexterity in software development is an imperative for global managers and developers. **C**

REFERENCES

1. Birkinshaw, J. and Gibson, C.B. Building ambidexterity into an organization. *MIT Sloan Management Review* 45, 4 (2004), 47–55.
2. Boehm, B.W. and Turner, R. *Balancing Agility and Discipline: A Guide for the Perplexed*. Addison-Wesley, 2004.
3. Cockburn, A. *Agile Software Development*. Addison-Wesley, Boston, MA, 2001.
4. Espinosa, J.A., Cummings, J.N., Wilson, J.M., and Pearce, B.M. Team boundary issues across multiple global firms. *J. Management Information Systems* 19, 4 (2003).
5. Fitzgerald, B., Russo, N.L., and O’Kane, T. Software development method tailoring at Motorola. *Commun. ACM* 46, 4 (Apr. 2003), 64–70.
6. Kraut, R.E. and Streeter, L.A. Coordination in software development. *Commun. ACM* 38, 3 (Mar. 1995), 69–81.
7. Krishna, S., Sahay, S., and Walsham, G. Managing cross-cultural issues in global software outsourcing. *Commun. ACM* 47, 4 (Apr. 2004), 62–66.
8. Nerur, S., Mahapatra, R., and Mangalaraj, G. Challenges of migrating to agile methodologies. *Commun. ACM* 48, 5 (May 2005), 73–78.
9. Orlikowski, W. Knowing in practice: Enacting a collective capability in distributed organizing. *Organization Science* 13, 3 (2002), 249–273.
10. Powell, A., Piccoli, G., and Ives, B. Virtual teams: A review of current literature and directions for future research. *Data Base for Advances in Information Systems* 35, 1 (2004), 6–36.
11. Reifer, D.J., Maurer, F., and Erdogmus, H. Scaling agile methods. *IEEE Software* 20, 4 (2003), 12–14.
12. Strauss, A. and Corbin, J. *Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory*. Thousand Oaks, London, U.K., 1998.

GWANHOO LEE (glee@american.edu) is an assistant professor of information technology and UPS scholar in the Kogod School of Business at American University, Washington, D.C.

WILLIAM DELONE (wdelone@american.edu) is a professor of information technology and the director of the Center for Information Technology and the Global Economy in the Kogod School of Business at American University, Washington, D.C.

J. ALBERTO ESPINOSA (alberto@american.edu) is an assistant professor of information technology and UPS scholar in the Kogod School of Business at American University, Washington, D.C.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.